Etapa 2 — Prueba completa del CRUD real

Esta guía valida la etapa 2 del MVP: CRUD básico de usuarios, creación/listado/borrado de tareas, asignación de tareas a usuarios, estados de tareas y relaciones `Usuario → tareas` y `Proyecto → tareas`.

> Nota: en el roadmap se nombran rutas como `GET /users`, `POST /users` y `DELETE /users`. En este backend real las rutas están bajo el prefijo `/api`, por ejemplo `GET /api/users`.

Requisitos

- Haber completado la etapa 1.
- PostgreSQL levantado con Docker Compose.
- Backend Spring Boot levantado en `http://localhost:8081`.
- `curl` disponible.
- Opcional: `jq` para extraer el token automáticamente.

1. Levantar PostgreSQL

Desde la carpeta raíz del MVP:

```bash
cd taskflow-mvp
docker compose up -d postgres
```

Verificá el estado del contenedor:

```bash

docker compose ps

```


2. Levantar la API

Desde la carpeta del backend:


```bash

cd taskflow-mvp/backend/taskflow-api/taskflow-api

./mvnw spring-boot:run

```


La API debe quedar disponible en:


```text

http://localhost:8081

```


3. Crear usuario base para autenticación

El endpoint de registro es público y sirve para crear un usuario inicial con password conocido:


```bash

curl -i -X POST http://localhost:8081/api/users/register \

 -H "Content-Type: application/json" \

 -d '{

   "name": "Tester Etapa 2",
```

```
   "email": "tester.etapa2@example.com",

   "password": "secret123"

 }'
```

Resultado esperado:

- HTTP `201 Created` si el usuario no existía.

- HTTP `409 Conflict` si ya lo habías creado antes; en ese caso podés seguir con el login del próximo paso.

4. Iniciar sesión y guardar el token JWT

Con `jq`:

```bash
TOKEN=$(curl -s -X POST http://localhost:8081/auth/login \

 -H "Content-Type: application/json" \

 -d '{

   "email": "tester.etapa2@example.com",

   "password": "secret123"

 }' | jq -r '.token')

echo "$TOKEN"
```

Sin `jq`, copiá el valor de `token` manualmente:

```bash
```

```
curl -i -X POST http://localhost:8081/auth/login \
 -H "Content-Type: application/json" \
 -d '{
   "email": "tester.etapa2@example.com",
   "password": "secret123"
 }'
```

Después exportalo:

```bash
TOKEN="PEGAR_TOKEN_ACA"
```

Todos los endpoints de CRUD de etapa 2 usan este header:

```bash
-H "Authorization: Bearer $TOKEN"
```

5. Día 3 — CRUD básico de usuarios

5.1 Listar usuarios — `GET /api/users`

```bash
curl -i http://localhost:8081/api/users \
 -H "Authorization: Bearer $TOKEN"
```

Resultado esperado: HTTP `200 OK` y un array JSON con usuarios.

5.2 Crear usuario — `POST /api/users`

```bash
curl -i -X POST http://localhost:8081/api/users \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d '{
   "name": "Usuario CRUD Etapa 2",
   "email": "usuario.crud.etapa2@example.com"
 }'
```

Resultado esperado:

- HTTP `201 Created`.
- JSON con `id`, `name`, `email`, `role` y `createdAt`.
- Guardá el `id` devuelto para probar `DELETE /api/users/{id}`.

Si usás `jq`, podés guardar el ID automáticamente:

```bash
CRUD_USER_ID=$(curl -s -X POST http://localhost:8081/api/users \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d '{
   "name": "Usuario CRUD Etapa 2 Temp",
```

```
   "email": "usuario.crud.temp.etapa2@example.com"
 }' | jq -r '.id')

echo "$CRUD_USER_ID"
```

5.3 Borrar usuario — `DELETE /api/users/{id}`

Usá un usuario temporal que no tenga proyectos ni tareas asociados:

```bash
curl -i -X DELETE http://localhost:8081/api/users/$CRUD_USER_ID \
 -H "Authorization: Bearer $TOKEN"
```

Resultado esperado: HTTP `204 No Content`.

Confirmá que ya no aparece:

```bash
curl -s http://localhost:8081/api/users \
 -H "Authorization: Bearer $TOKEN"
```

6. Preparar IDs para tareas y proyectos

Para probar relaciones reales, conviene crear un usuario asignado y un proyecto.

6.1 Crear usuario asignado a tareas

```bash
ASSIGNEE_ID=$(curl -s -X POST http://localhost:8081/api/users \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d '{
   "name": "Dev Asignado Etapa 2",
   "email": "dev.asignado.etapa2@example.com"
 }' | jq -r '.id')

echo "$ASSIGNEE_ID"
```

Si no usás `jq`, ejecutá el mismo `POST`, copiá el `id` de la respuesta y exportalo:

```bash
ASSIGNEE_ID="PEGAR_ID_USUARIO_ASIGNADO"
```

6.2 Crear proyecto

```bash
PROJECT_ID=$(curl -s -X POST http://localhost:8081/api/projects \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d "{
   \"name\": \"Proyecto Etapa 2\",
   \"description\": \"Proyecto para validar relaciones con tareas\",
```

```
   \"ownerId\": $ASSIGNEE_ID

 }" | jq -r '.id')


echo "$PROJECT_ID"

```


Sin `jq`, copiá el `id` devuelto por `POST /api/projects` y exportalo:


```bash

PROJECT_ID="PEGAR_ID_PROYECTO"

```


7. Día 4 — CRUD básico de tareas

7.1 Crear tarea PENDING asignada a usuario y proyecto


```bash

TASK_PENDING_ID=$(curl -s -X POST http://localhost:8081/api/tasks \

 -H "Authorization: Bearer $TOKEN" \

 -H "Content-Type: application/json" \

 -d "{

   \"title\": \"Diseñar tablero Etapa 2\",

   \"description\": \"Primera tarea real del CRUD\",

   \"status\": \"PENDING\",

   \"priority\": \"HIGH\",

   \"dueDate\": \"2026-05-10\",

   \"projectId\": $PROJECT_ID,

   \"assignedToUserId\": $ASSIGNEE_ID
```

```
 }" | jq -r '.id')


echo "$TASK_PENDING_ID"
```

Resultado esperado: la tarea se crea con `status: "PENDING"`, `projectId` y `assignedToUserId`.

7.2 Crear tarea IN_PROGRESS

```bash
curl -i -X POST http://localhost:8081/api/tasks \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d "{
   \"title\": \"Implementar servicio de tareas\",
   \"description\": \"Validar estado en progreso\",
   \"status\": \"IN_PROGRESS\",
   \"priority\": \"MEDIUM\",
   \"dueDate\": \"2026-05-11\",
   \"projectId\": $PROJECT_ID,
   \"assignedToUserId\": $ASSIGNEE_ID
 }"
```

Resultado esperado: HTTP `201 Created` y `status: "IN_PROGRESS"`.

7.3 Crear tarea DONE

```bash
curl -i -X POST http://localhost:8081/api/tasks \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d "{
   \"title\": \"Cerrar checklist Etapa 2\",
   \"description\": \"Validar estado terminado\",
   \"status\": \"DONE\",
   \"priority\": \"LOW\",
   \"dueDate\": \"2026-05-12\",
   \"projectId\": $PROJECT_ID,
   \"assignedToUserId\": $ASSIGNEE_ID
 }"
```

Resultado esperado: HTTP `201 Created` y `status: "DONE"`.

7.4 Listar tareas — `GET /api/tasks`

```bash
curl -i http://localhost:8081/api/tasks \
 -H "Authorization: Bearer $TOKEN"
```

Resultado esperado: HTTP `200 OK` y un array con las tareas creadas.

7.5 Borrar tarea — `DELETE /api/tasks/{id}`

```bash
```

```
curl -i -X DELETE http://localhost:8081/api/tasks/$TASK_PENDING_ID \
 -H "Authorization: Bearer $TOKEN"
```

Resultado esperado: HTTP `204 No Content`.

8. Día 5 — Relaciones reales

8.1 Usuario → tareas

```bash
curl -i http://localhost:8081/api/tasks/by-user/$ASSIGNEE_ID \
 -H "Authorization: Bearer $TOKEN"
```

Resultado esperado: HTTP `200 OK` y tareas cuyo `assignedToUserId` sea `$ASSIGNEE_ID`.

8.2 Proyecto → tareas

```bash
curl -i http://localhost:8081/api/tasks/by-project/$PROJECT_ID \
 -H "Authorization: Bearer $TOKEN"
```

Resultado esperado: HTTP `200 OK` y tareas cuyo `projectId` sea `$PROJECT_ID`.

8.3 Validación directa en PostgreSQL

```bash

docker exec -it taskflow-db psql -U taskflow_user -d taskflow -c "select t.id, t.title, t.status, t.project_id, t.assigned_to_id, u.email as assigned_email, p.name as project_name from tasks t left join users u on u.id = t.assigned_to_id left join project p on p.id = t.project_id order by t.id;"

```


Resultado esperado: filas en `tasks` relacionadas con el usuario asignado y el proyecto creado.

> Si tu tabla de proyectos aparece como `projects` en vez de `project`, ajustá el último `join` a `left join projects p on p.id = t.project_id`.

9. Validaciones útiles de etapa 2

9.1 Crear tarea con usuario inexistente


```bash

curl -i -X POST http://localhost:8081/api/tasks \

 -H "Authorization: Bearer $TOKEN" \

 -H "Content-Type: application/json" \

 -d "{

   \"title\": \"Tarea con usuario inexistente\",

   \"description\": \"Debe fallar\",

   \"status\": \"PENDING\",

   \"priority\": \"LOW\",

   \"dueDate\": \"2026-05-13\",

   \"projectId\": $PROJECT_ID,

   \"assignedToUserId\": 999999

 }"

```

Resultado esperado: HTTP `404 Not Found` con mensaje `Assigned user not found`.

9.2 Crear tarea con proyecto inexistente

```bash
curl -i -X POST http://localhost:8081/api/tasks \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d "{
   \"title\": \"Tarea con proyecto inexistente\",
   \"description\": \"Debe fallar\",
   \"status\": \"PENDING\",
   \"priority\": \"LOW\",
   \"dueDate\": \"2026-05-13\",
   \"projectId\": 999999,
   \"assignedToUserId\": $ASSIGNEE_ID
 }"
```

Resultado esperado: HTTP `404 Not Found` con mensaje `Project not found`.

9.3 Crear tarea con estado inválido

```bash
curl -i -X POST http://localhost:8081/api/tasks \
 -H "Authorization: Bearer $TOKEN" \
 -H "Content-Type: application/json" \
 -d "{
```

```
   \"title\": \"Tarea con estado inválido\",

   \"description\": \"Debe fallar\",

   \"status\": \"BLOCKED\",

   \"priority\": \"LOW\",

   \"dueDate\": \"2026-05-13\",

   \"projectId\": $PROJECT_ID,

   \"assignedToUserId\": $ASSIGNEE_ID

 }"

```


Resultado esperado: HTTP `400 Bad Request`.

10. Checklist de etapa 2

- [ ] `GET /api/users` lista usuarios autenticados.

- [ ] `POST /api/users` crea usuarios autenticados.

- [ ] `DELETE /api/users/{id}` borra usuarios sin relaciones activas.

- [ ] `POST /api/projects` crea un proyecto con dueño.

- [ ] `POST /api/tasks` crea tareas asignadas a usuario y proyecto.

- [ ] Las tareas aceptan `PENDING`, `IN_PROGRESS` y `DONE`.

- [ ] `GET /api/tasks` lista tareas.

- [ ] `DELETE /api/tasks/{id}` borra tareas.

- [ ] `GET /api/tasks/by-user/{userId}` valida la relación `Usuario → tareas`.

- [ ] `GET /api/tasks/by-project/{projectId}` valida la relación `Proyecto → tareas`.

- [ ] Crear tareas con usuario/proyecto inexistente devuelve error.

11. Objetivo final

```
Si todos los checks pasan, la etapa 2 cumple el objetivo: sistema funcional básico con
usuarios, proyectos y tareas relacionados.
```